

Taft, R., Sharif, I., Matei, A., VanBenschoten, N., Lewis, J., Grieger, T., Niemi, K., Woods, A., Birzin, A., Poss, R., Bardea, P., Ranade, A., Darnell, B., Gruneir, B., Jaffray, J., Zhang, L., & Mattis, P. (2020). CockroachDB: The resilient geo-distributed SQL database. Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 1493–1509. <https://doi.org/10.1145/3318464.3386134>

El artículo aborda el problema de administrar bases de datos SQL distribuidas geográficamente que deben atender grandes cantidades de usuarios manteniendo alta disponibilidad y consistencia. Los sistemas tradicionales pueden presentar dificultades cuando los datos y usuarios se encuentran distribuidos entre diferentes regiones.

Los autores presentan CockroachDB, un sistema de gestión de bases de datos SQL distribuido diseñado para cargas de trabajo OLTP globales. La propuesta utiliza replicación, recuperación automática, distribución geográfica de los datos y un modelo de transacciones distribuido que mantiene consistencia, además de proporcionar una interfaz SQL estándar.

Los experimentos muestran que CockroachDB puede escalar horizontalmente y mantener un alto rendimiento en configuraciones distribuidas. Por ejemplo, en las pruebas TPC-C alcanzó hasta 1,245,462 tpmC con 81 nodos, con una eficiencia reportada de 98.8 %. Además, en las pruebas con YCSB presentó mayor rendimiento en varias cargas de trabajo y menores latencias que Spanner bajo las condiciones evaluadas.

Este artículo nos ayuda a comprender que el diseño de una base de datos no solamente consiste en crear tablas y relaciones, sino también en considerar aspectos como consistencia, disponibilidad y crecimiento del sistema. Para nuestro proyecto, nos permitira pensar en cómo diseñar la base de datos de manera que pueda manejar un aumento de información y usuarios sin perder integridad ni rendimiento.

Verbitski, A., Gupta, A., Saha, D., Corey, J., Gupta, K., Brahmadesam, M., Mittal, R., Krishnamurthy, S., Maurice, S., Kharatishvili, T., & Bao, X. (2018). Amazon Aurora: On avoiding distributed consensus for I/Os, commits, and membership changes. *Proceedings of the 2018 International Conference on Management of Data*, 789–796. <https://doi.org/10.1145/3183713.3196937>

El artículo aborda el problema de mantener una base de datos relacional de alto rendimiento y disponibilidad en un entorno distribuido, donde las operaciones de almacenamiento, replicación y recuperación pueden generar costos y retrasos. Los autores buscan reducir la dependencia de mecanismos tradicionales de consenso distribuido que pueden incrementar el tráfico de red y la latencia.

Los autores presentan el diseño de Amazon Aurora, una base de datos relacional orientada a la nube que separa el procesamiento de la base de datos del almacenamiento. Aurora utiliza un servicio de almacenamiento distribuido y procesa los registros de cambios en dicho servicio, evitando utilizar consenso distribuido en muchas operaciones de E/S, confirmaciones y cambios de membresía.

La propuesta se basa además en invariantes del sistema y estado transitorio local, con el objetivo de mantener la consistencia y tolerancia a fallos reduciendo la necesidad de coordinación entre los diferentes componentes.

Los autores muestran que el diseño de Aurora permite reducir la coordinación distribuida y los costos asociados al almacenamiento, manteniendo alta disponibilidad, durabilidad y rendimiento. La arquitectura permite realizar recuperación ante fallos y realizar failover hacia réplicas sin pérdida de datos, mientras que la reducción del consenso distribuido disminuye el tráfico de red y la variabilidad de la latencia.

Este artículo nos aporta conocimientos sobre la importancia de considerar la disponibilidad, recuperación ante fallos y organización del almacenamiento al diseñar una base de datos. Para nuestro proyecto, nos permite identificar que además de definir correctamente las tablas y relaciones, es importante pensar en cómo se conservarían y recuperarían los datos ante posibles fallas del sistema.

Nathan, S., Govindarajan, C., Saraf, A., Sethi, M., & Jayachandran, P. (2019). Blockchain meets database: Design and implementation of a blockchain relational database. *Proceedings of the VLDB Endowment*, 12(11), 1539–1552.
<https://doi.org/10.14778/3342263.3342632>

El artículo aborda el problema de construir una base de datos relacional descentralizada y replicada en la que los diferentes nodos no necesitan confiar unos en otros. A diferencia de los sistemas tradicionales de replicación, los nodos deben ejecutar las transacciones de manera independiente y llegar a un acuerdo sobre el orden de ejecución y confirmación para conservar la consistencia

Los autores proponen una base de datos relacional con características de blockchain, implementada sobre PostgreSQL. Presentan dos mecanismos: uno en el que el orden de confirmación de las transacciones se determina antes de ejecutarlas y otro en el que las transacciones se ejecutan mientras el orden se determina en paralelo. Para mantener la consistencia entre las réplicas utilizan Serializable Snapshot Isolation (SSI) y una variante basada en la altura del bloque

Los autores implementan y evalúan ambos enfoques sobre PostgreSQL mediante experimentos de rendimiento. El trabajo demuestra la viabilidad de combinar las características de una base de datos relacional con mecanismos de replicación y consenso descentralizado propios de blockchain, manteniendo un orden serializable y consistente entre los diferentes nodos.

Este artículo nos permite comprender la importancia de mantener la consistencia e integridad de los datos cuando existen varias copias de una misma base de datos. Para nuestro proyecto, aporta ideas sobre la importancia de diseñar correctamente las relaciones y transacciones para evitar inconsistencias cuando diferentes usuarios o procesos modifican la información simultáneamente.

Los tres artículos tienen como punto en común el estudio de sistemas de bases de datos distribuidos, especialmente los problemas relacionados con la replicación, consistencia, disponibilidad y coordinación entre diferentes nodos. Sin embargo, cada uno aborda estas cuestiones desde una perspectiva diferente. CockroachDB propone una base de datos SQL distribuida geográficamente, buscando que los datos puedan mantenerse disponibles y consistentes mientras el sistema escala entre diferentes ubicaciones. Amazon Aurora, por su parte, se concentra en una arquitectura de almacenamiento distribuido para bases de datos relacionales, buscando reducir la necesidad de coordinación distribuida y mejorar aspectos como rendimiento, disponibilidad y recuperación ante fallos. El tercer artículo, Blockchain Meets Database, lleva la distribución hacia un escenario de nodos que no necesariamente confían entre sí y combina una base de datos relacional con mecanismos de consenso propios de blockchain. Así, mientras CockroachDB enfatiza la escalabilidad y distribución geográfica, Aurora pone mayor énfasis en la eficiencia del almacenamiento y la reducción de coordinación, y Nathan et al. se enfocan en la descentralización, consenso y consistencia entre réplicas